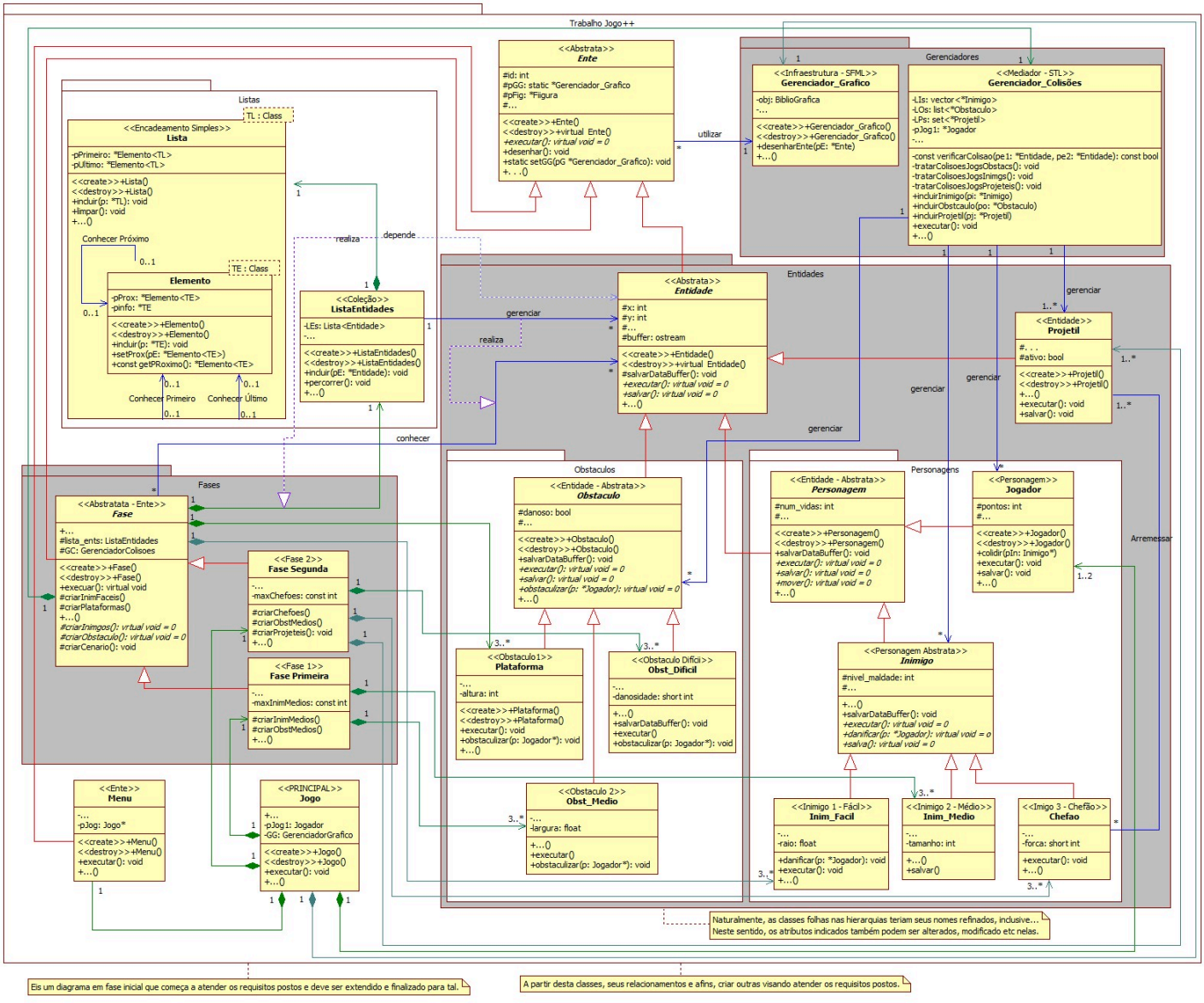

Trabalho Jogo++
Listas
Entidades
Gerenciadores
Fases
Obstaculos
Personagens
<<Abstrata>>
Ente
#id: int
#pGG: static *Gerenciador_Grafico
#pFig: *Figura
#...
<<create>>+Ente()
<<destroy>>+virtual Ente()
+executar(): virtual void = 0
+desenhar(): void
+static setGG(pG *Gerenciador_Grafico): void
+...()
utilizar
<<Infraestrutura - SFML>>
Gerenciador_Grafico
-obj: BiblioGrafica
-...
<<create>>+Gerenciador_Grafico()
<<destroy>>+Gerenciador_Grafico()
+desenharEnte(pE: *Ente)
+...()
<<Mediador - STL>>
Gerenciador_Colisões
-lIs: vector<*Inimigo>
-lOs: list<*Obstaculo>
-lPs: set<*Projetil>
-pJog1: *Jogador
-...
-const verificarColisao(pe1: *Entidade, pe2: *Entidade): const bool
-tratarColisoesJogsObstacs(): void
-tratarColisoesJogsInimgs(): void
-tratarColisoesJogsProjeteis(): void
+incluirInimigo(pi: *Inimigo)
+incluirObstculo(po: *Obstaculo)
+incluirProjetil(pj: *Projetil)
+executar(): void
+...()
<<Encadeamento Simples>>
Lista
-pPrimeiro: *Elemento<TL>
-pUltimo: *Elemento<TL>
<<create>>+Lista()
<<destroy>>+Lista()
+incluir(p: *TL): void
+limpar(): void
+...()
TL : Class
Conhecer Próximo
Elemento
-pProx: *Elemento<TE>
-pInfo: *TE
<<create>>+Elemento()
<<destroy>>+Elemento()
+incluir(p: *TE): void
+setProx(pE: *Elemento<TE>)
+const getProximo(): *Elemento<TE>
TE : Class
Conhecer Primeiro
Conhecer Último
realiza
depende
<<Coleção>>
ListaEntidades
-lEs: Lista<Entidade>
-...
<<create>>+ListaEntidades()
<<destroy>>+ListaEntidades()
+incluir(pE: *Entidade): void
+percorrer(): void
+...()
gerenciar
realiza
<<Abstrata>>
Entidade
#x: int
#y: int
#...
#buffer: ostream
<<create>>+Entidade()
<<destroy>>+virtual Entidade()
#salvarDataBuffer(): void
+executar(): virtual void = 0
+salvar(): virtual void = 0
+...()
<<Entidade>>
Projetil
#...
#ativo: bool
<<create>>+Projetil()
<<destroy>>+Projetil()
+...()
+executar(): void
+salvar(): void
gerenciar
gerenciar
gerenciar
conhecer
<<Abstratata - Ente>>
Fase
+...
#lista_ents: ListaEntidades
#GC: GerenciadorColisoes
<<create>>+Fase()
<<destroy>>+Fase()
+execuar(): virtual void
#criarInimFaceis()
#criarPlataformas()
+...()
#criarInimgos(): virtual void = 0
#criarObstaculo(): virtual void = 0
#criarCenario(): void
<<Fase 2>>
Fase Segunda
-...
-maxChefoes: const int
#criarChefoes()
#criarObstMedios()
#criarProjeteis(): void
+...()
<<Fase 1>>
Fase Primeira
-...
-maxInimMedios: const int
#criarInimMedios()
#criarObstMedios()
+...()
<<Entidade - Abstrata>>
Obstaculo
#danoso: bool
#...
<<create>>+Obstaculo()
<<destroy>>+Obstaculo()
+salvarDataBuffer(): void
+executar(): virtual void = 0
+salvar(): virtual void = 0
+obstaculizar(p: *Jogador): virtual void = 0
+...()
<<Obstaculo1>>
Plataforma
-...
-altura: int
<<create>>+Plataforma()
<<destroy>>+Plataforma()
+executar(): void
+obstaculizar(p: Jogador*): void
+...()
<<Obstaculo Difici>>
Obst_Dificil
-...
-danosidade: short int
+...()
+salvarDataBuffer(): void
+executar()
+obstaculizar(p: Jogador*): void
<<Obstaculo 2>>
Obst_Medio
-...
-largura: float
+...()
+executar()
+obstaculizar(p: Jogador*): void
<<Entidade - Abstrata>>
Personagem
#num_vidas: int
#...
<<create>>+Personagem()
<<destroy>>+Personagem()
+salvarDataBuffer(): void
+executar(): virtual void = 0
+salvar(): virtual void = 0
+mover(): virtual void = 0
+...()
<<Personagem>>
Jogador
#pontos: int
#...
<<create>>+Jogador()
<<destroy>>+Jogador()
+colidir(pIn: Inimigo*)
+executar(): void
+salvar(): void
+...()
Arremessar
<<Personagem Abstrata>>
Inimigo
#nivel_maldade: int
#...
+...()
+salvarDataBuffer(): void
+executar(): virtual void = 0
+danificar(p: *Jogador): virtual void = 0
+salva(): virtual void = 0
<<Inimigo 1 - Fácil>>
Inim_Facil
-...
-raio: float
+danificar(p: *Jogador): void
+executar(): void
+...()
<<Inimigo 2 - Médio>>
Inim_Medio
-...
-tamanho: int
+...()
+salvar()
<<Imigo 3 - Chefão>>
Chefao
-...
-forca: short int
+executar(): void
+...()
<<Ente>>
Menu
-...
-pJog: Jogo*
<<create>>+Menu()
<<destroy>>+Menu()
+executar(): void
+...()
<<PRINCIPAL>>
Jogo
+...
-pJog1: Jogador
-GG: GerenciadorGrafico
<<create>>+Jogo()
<<destroy>>+Jogo()
+executar(): void
+...()
Naturalmente, as classes folhas nas hierarquias teriam seus nomes refinados, inclusive...
Neste sentido, os atributos indicados também podem ser alterados, modificado etc nelas.
Eis um diagrama em fase inicial que começa a atender os requisitos postos e deve ser extendido e finalizado para tal.
A partir desta classes, seus relacionamentos e afins, criar outras visando atender os requisitos postos.